react-agent

August 16, 2025

[1]:
```
# ReAct agent (Reasoning and Acting) from scratch using GPT-4
# ReAct works on the following principle
# Thought -> Action -> Observation (Pause) loop to answer user queries␣
 ↪effectively.
# The example is build using GPT-4
```

[2]:
```
#  import the necessary libraries and set up the OpenAI client.
from openai import OpenAI
import re

from dotenv import load_dotenv
_ = load_dotenv()

client = OpenAI()
```

[3]:
```
# Agent class that will handle conversations with the user and interact with␣
 ↪the OpenAI API.

class Agent:
    """
    A class representing an AI agent that can engage in conversations using␣
 ↪OpenAI's API.
    """

    def __init__(self, system=""):
        """
        Initialize the agent with an optional system message.

        Args:
            system (str): The system message to set the context for the agent.
        """
        self.system = system
        self.messages = []
        if self.system:
            # If a system message is provided, add it to the message history
            self.messages.append({"role": "system", "content": system})
```

```
    def __call__(self, message):
        """
        Allow the agent to be called directly with a message.

        Args:
            message (str): The user's input message.

        Returns:
            str: The agent's response to the input message.
        """
        # Add the user's message to the conversation history
        self.messages.append({"role": "user", "content": message})
        # Execute the conversation and get the result
        result = self.execute()
        # Add the assistant's response to the conversation history
        self.messages.append({"role": "assistant", "content": result})
        return result

    def execute(self):
        """
        Execute the conversation by sending the entire conversation history to␣
 ↪the OpenAI API.

        Returns:
            str: The content of the model's response.
        """
        # Send the entire conversation history to the OpenAI API
        completion = client.chat.completions.create(
                        model="gpt-4o-mini",
                        temperature=0,
                        messages=self.messages)
        # Return the content of the model's response
        return completion.choices[0].message.content
```

[4]:
```
# define a prompt that sets the context and behavior for the agent. The agent␣
 ↪operates in a loop of
# Thought, Action, PAUSE, and Observation, and finally outputs an Answer.

# Define a prompt for the agent
prompt = """
You run in a loop of Thought, Action, PAUSE, Observation.
At the end of the loop you output an Answer.
Use Thought to describe your thoughts about the question you have been asked.
Use Action to run one of the actions available to you - then return PAUSE.
Observation will be the result of running those actions.

Your available actions are:
```

```
calculate_total_price:
e.g. calculate_total_price: apple: 2, banana: 3
Runs a calculation for the total price based on the quantity and prices of the␣
 ↪fruits.

get_fruit_price:
e.g. get_fruit_price: apple
returns the price of the fruit when given its name.

Example session:

Question: What is the total price for 2 apples and 3 bananas?
Thought: I should calculate the total price by getting the price of each fruit␣
 ↪and summing them up.
Action: get_fruit_price: apple
PAUSE

Observation: The price of an apple is $1.5.

Action: get_fruit_price: banana
PAUSE

Observation: The price of a banana is $1.2.

Action: calculate_total_price: apple: 2, banana: 3
PAUSE

You then output:

Answer: The total price for 2 apples and 3 bananas is $6.6.
""".strip()
```

[5]:
```python
# define functions that the agent can use to perform actions
# Price lookup for fruits
fruit_prices = {
    "apple": 1.5,
    "banana": 1.2,
    "orange": 1.3,
    "grapes": 2.0
}

# Function to calculate the price of a specific fruit
def get_fruit_price(fruit):
    if fruit in fruit_prices:
        return f"The price of a {fruit} is ${fruit_prices[fruit]}"
    else:
```

```
        return f"Sorry, I don't know the price of {fruit}."

# Function to calculate total price based on quantities
def calculate_total_price(fruits):
    total = 0.0
    fruit_list = fruits.split(", ")
    for item in fruit_list:
        fruit, quantity = item.split(": ")
        quantity = int(quantity)
        if fruit in fruit_prices:
            total += fruit_prices[fruit] * quantity
        else:
            return f"Sorry, I don't have the price of {fruit}."
    return f"The total price is ${total:.2f}"

# Mapping actions to functions
known_actions = {
    "get_fruit_price": get_fruit_price,
    "calculate_total_price": calculate_total_price
}
```

[6]:
```
# create an instance of the Agent class with the defined prompt.
react_agent = Agent(system=prompt)
```

[7]:
```
# define a function query that takes a question, sends it to the agent, parses␣
 ↪the agent's response for actions,
# and executes the corresponding functions.

# Run a query
action_re = re.compile(r'^Action: (\w+): (.*)$')   # python regular expression␣
 ↪to select action

def query(question):
    bot = Agent(prompt)
    result = bot(question)
    print(result)
    actions = [
        action_re.match(a)
        for a in result.split('\n')
        if action_re.match(a)
    ]
    if actions:
        action, action_input = actions[0].groups()
        if action not in known_actions:
            raise Exception(f"Unknown action: {action}: {action_input}")
        print(f" -- running {action} {action_input}")
        observation = known_actions[action](action_input)
```

```
        print("Observation:", observation)
    else:
        return
```

0.1 Running Queries

[8]:
```
query("What is the price of a banana?")
```

```
Thought: I need to find out the price of a banana by retrieving its current
price.
Action: get_fruit_price: banana
PAUSE
 -- running get_fruit_price banana
Observation: The price of a banana is $1.2
```

[9]:
```
query("What is the price of 2 bananas?")
```

```
Thought: To find the price of 2 bananas, I need to first get the price of a
single banana and then multiply it by 2.
Action: get_fruit_price: banana
PAUSE
 -- running get_fruit_price banana
Observation: The price of a banana is $1.2
```

[10]:
```
query("What is the price of 2 bananas and 3 oranges?")
```

```
Thought: I need to find the price of each fruit, specifically bananas and
oranges, to calculate the total price for 2 bananas and 3 oranges.
Action: get_fruit_price: banana
PAUSE
 -- running get_fruit_price banana
Observation: The price of a banana is $1.2
```

1 Handling Multiple Turns with Loops

[11]:
```
# To allow the agent to perform multiple actions in a loop (e.g., get prices␣
 ↪before calculating total),
# we modify the query function to handle multiple turns until the agent outputs␣
 ↪an Answer.

# Run a query
action_re = re.compile(r'^Action: (\w+): (.*)$')   # python regular expression␣
 ↪to select action

def query(question, max_turns=5):
    i = 0
```

```
    bot = Agent(prompt)
    next_prompt = question
    while i < max_turns:
        i += 1
        result = bot(next_prompt)
        print(result)
        actions = [
            action_re.match(a)
            for a in result.split('\n')
            if action_re.match(a)
        ]
        if actions:
            action, action_input = actions[0].groups()
            if action not in known_actions:
                raise Exception(f"Unknown action: {action}: {action_input}")
            print(f" -- running {action} {action_input}")
            observation = known_actions[action](action_input)
            print("Observation:", observation)
            next_prompt = f"Observation: {observation}"
        else:
            return
```

1.1 Running Queries with Multiple Turns

[12]:
```
query("What is the price of 2 bananas?")
```

```
Thought: To find the price of 2 bananas, I need to first get the price of a
single banana and then multiply it by 2.
Action: get_fruit_price: banana
PAUSE
 -- running get_fruit_price banana
Observation: The price of a banana is $1.2
Action: calculate_total_price: banana: 2
PAUSE
 -- running calculate_total_price banana: 2
Observation: The total price is $2.40
Answer: The price of 2 bananas is $2.40.
```

[13]:
```
query("If i bought 10 apples, 10 bananas, and 2 oranges, how much would it cost?
 ↪")
```

```
Thought: To find the total cost, I need to get the prices of apples, bananas,
and oranges, and then calculate the total based on the quantities provided.

Action: get_fruit_price: apple
PAUSE
 -- running get_fruit_price apple
```

```
Observation: The price of a apple is $1.5
Action: get_fruit_price: banana
PAUSE
 -- running get_fruit_price banana
Observation: The price of a banana is $1.2
Action: get_fruit_price: orange
PAUSE
 -- running get_fruit_price orange
Observation: The price of a orange is $1.3
Action: calculate_total_price: apple: 10, banana: 10, orange: 2
PAUSE
 -- running calculate_total_price apple: 10, banana: 10, orange: 2
Observation: The total price is $29.60
Answer: The total price for 10 apples, 10 bananas, and 2 oranges is $29.60.
```

[ ]: